WHITE PAPER

VERIFIABLE CLASSIFICATION DOWNGRADING THROUGH INFORMATION-DESTROYING TRANSFORMS

Authors: Tom Tang, Fei Jia
Organization: Edge Vector Foundation
Date: March 16, 2026

ABSTRACT

We present a formal mechanism for controlled downgrading of data classification levels through publicly visible, automatically verified transforms. In FoldDB's lattice-based security label system, information flow is strictly upward: data at a given classification level can never be exposed at a lower level. This paper relaxes that constraint in a principled way. We introduce *downgrading transforms* — statistical aggregation functions (means, medians, counts, histograms) that provably destroy information about individual records. The key insight is that not all information loss is equal: a transform that computes the average of 1,000 salaries genuinely destroys individual salary data, while a transform that "anonymizes" a unique record by removing one field may not. We formalize this distinction through an *empirical verification protocol* that tests each transform with synthetic inputs to measure the degree of information destruction, and a *minimum population threshold* that ensures aggregation operates over enough records to prevent reconstruction. Transforms whose information loss cannot be empirically verified do not receive downgrading privileges and remain at the source classification level. All downgrading transforms are registered in a public registry with their verification results, enabling any data owner to inspect and audit the downgrading chain before exposing their data to it.

1. INTRODUCTION

1.1 The Downgrading Problem

FoldDB enforces lattice-based security labels on all data fields. Each field carries a label ℓ from a partially ordered set $(\mathcal{L}, \sqsubseteq)$, and transforms are constrained so that outputs must carry labels at least as high as their inputs: $\ell_{\text{in}} \sqsubseteq \ell_{\text{out}}$. This is the standard Bell–LaPadula

"no write down" property, and it prevents sensitive data from being laundered into lower-classified outputs.

This is correct and necessary for general transforms. A function that reformats a Social Security number or translates a medical diagnosis into another language does not reduce the sensitivity of the data — the same information is fully recoverable from the output.

However, certain transforms genuinely destroy information. The arithmetic mean of 10,000 salary values contains far less information than the individual salaries. A histogram of age ranges reveals distributional properties while making individual ages unrecoverable. A count of records matching a criterion reveals only cardinality. These *statistical aggregations* produce outputs that are, in a formal information-theoretic sense, less sensitive than their inputs.

The current system has no mechanism to express this. A mean computed from SECRET-labeled salaries must itself carry the SECRET label, even though the mean reveals no individual salary. This forces a choice between two undesirable outcomes:

1. **Over-classification**: Aggregated statistics carry the same label as raw data, making them inaccessible to anyone who cannot access the raw data. This defeats the purpose of aggregation.
2. **Manual reclassification**: A human administrator manually assigns a lower label to aggregated outputs. This is error-prone, unauditable, and introduces a trust dependency on the administrator's judgment.

1.2 Our Contribution

We introduce a third option: *verified automatic downgrading*. A transform can receive a lower output label than its input label if and only if:

1. The transform is publicly visible (registered in the Universal Transform Registry).
2. The transform's information destruction is empirically verified through automated testing.
3. The degree of downgrading is bounded by the measured information loss.
4. The transform operates on a population above a minimum size threshold.

Transforms whose information loss cannot be empirically verified receive no downgrading. This is the critical safety property: *unverifiable information loss does not get downgraded*.

2. DATA CLASSIFICATION MODEL

2.1 Two-Dimensional Classification

Every field in FoldDB carries a classification label with two independent dimensions:

Definition 1 (Classification Label)**.** *A classification label $\ell = (s, d)$ consists of:*

- *A **sensitivity level** $s \in \{0, 1, 2, \ldots, s_{\max}\}$, where 0 is public and higher values indicate greater sensitivity.*
- *A **data domain** $d \in \mathcal{D}$, a categorical tag identifying the type of data (e.g., `financial`, `medical`, `identity`, `behavioral`, `location`).*

The sensitivity level determines *how restricted* access is. The data domain determines *what kind* of restriction applies. Two fields may have the same sensitivity level but different domains: an individual's salary (`financial`, level 3) and their diagnosis (`medical`, level 3) are equally sensitive but governed by different regulatory regimes and different aggregation properties.

Definition 2 (Label Ordering)**.** *Labels are partially ordered by sensitivity within the same domain:*

$$(s_1, d) \sqsubseteq (s_2, d) \iff s_1 \leq s_2$$

Labels with different domains are incomparable: (s_1, d_1) and (s_2, d_2) with $d_1 \neq d_2$ have no ordering relation. Cross-domain information flow requires explicit authorization.

2.2 Standard Sensitivity Levels

Level	Name	Description
0	Public	Freely distributable. No access restrictions.
1	Internal	Not sensitive but not intended for public release. Organizational context.
2	Confidential	Business-sensitive. Competitive or operational value.
3	Restricted	Personally identifiable or individually attributable data.
4	Highly Restricted	Regulated data (HIPAA, financial records, biometric identifiers).

These levels are defaults. Data owners may define custom levels for their domain.

3. DOWNGRADING TRANSFORMS

3.1 Definition

Definition 3 (Downgrading Transform)**.** *A downgrading transform $T_\downarrow$ is a function $T_\downarrow : \{V_1, \ldots, V_n\} \to V'$ that:*

1. *Takes as input a* ***set*** *of values (not a single value) from fields sharing classification (s, d).*
2. *Produces an output V' classified at (s', d) where $s' < s$.*
3. *Has been verified to destroy sufficient information that individual input values cannot be recovered from the output.*

The key distinction from ordinary transforms: downgrading transforms operate over *populations*, not individual records. A transform on a single record cannot downgrade, because there is no population to hide within.

3.2 Approved Aggregation Classes

The following classes of transforms are candidates for downgrading verification:

Class	**Examples**	**Max Downgrade**
Central tendency	Mean, median, mode	-2 levels
Dispersion	Std. deviation, variance, IQR	-2 levels
Distribution	Histogram, percentile bins, CDF	-1 level
Counting	Count, count distinct, ratio	-2 levels
Thresholding	Above/below threshold, range membership	-2 levels
Bucketing	k-anonymity grouping, range binning	-1 level

The "Max Downgrade" column indicates the maximum number of sensitivity levels the output can be reduced. The actual downgrade achieved depends on verification results and population size.

3.3 Non-Downgrading Transforms

The following transforms do **not** qualify for downgrading, regardless of how they are constructed:

- **Formatting changes**: converting units, changing date formats, translating text. The same information is preserved.
- **Subset selection**: filtering, sampling individual records. A single record at level 3 remains at level 3 even if selected from a larger set.
- **Encryption/hashing of individual values**: hashing a Social Security number does not reduce its sensitivity — the hash is a pseudonym for the same individual.
- **Redaction without aggregation**: removing one field from a record does not downgrade the remaining fields if the record is still individually attributable.

The principle: if the output can be traced back to a specific individual's input, no downgrading occurs.

4. VERIFICATION PROTOCOL

The verification protocol determines whether a proposed downgrading transform actually destroys sufficient information to justify a lower classification.

4.1 Overview

Verification is performed at transform registration time. The registrant submits the transform along with its claimed downgrade level. The system runs an automated test suite against the transform and either accepts the claimed downgrade, assigns a lesser downgrade, or rejects downgrading entirely.

4.2 Test Procedure

For a transform $T_\downarrow$ claiming to downgrade from level s to level s' over inputs of type τ:

Step 1: Synthetic population generation. Generate k synthetic populations $P_1, \ldots, P_k$, each of size $n \geq n_{\min}$ (the minimum population threshold), with values drawn from the type-appropriate distribution for τ.

Step 2: Output computation. For each population P_i, compute $o_i = T_\downarrow(P_i)$.

Step 3: Reconstruction attack. For each (P_i, o_i) pair, attempt to reconstruct individual input values from o_i:

(a) **Exact inversion**: Given o_i and knowledge of all inputs except one, can the missing input be computed? For a mean of n values, this is trivially possible ($x_{\text{missing}} = n \cdot \bar{x} - \sum x_{\text{known}}$). For a median of n values with $n > 2$, it is not.

(b) **Bounded inference**: Given o_i, can any individual input be constrained to a range smaller than ϵ fraction of the input domain? For a mean, each input is constrained to $[\bar{x} - R, \bar{x} + R]$ where R is the range of the data; the constraint tightens as n decreases.

(c) **Distinguishability**: Given o_i and two candidate populations P_i, P_i' that differ in exactly one element, can an observer determine which population produced o_i? This is the standard differential-privacy-style distinguishability test.

Step 4: Scoring. The verification score combines the results:

$$\text{score} = \begin{cases} 0 & \text{if exact inversion succeeds (any } n\text{-1 attack works)} \\ 1 & \text{if bounded inference constrains any input to} < 50\% \text{ of domain} \\ 2 & \text{if distinguishability ratio} < 1.5 \text{ and no bounded inference} < 50\% \\ 3 & \text{if distinguishability ratio} < 1.1 \text{ and no bounded inference} < 25\% \end{cases}$$

Step 5: Downgrade assignment. The granted downgrade is $\min(\text{score}, s - s')$. If the score is 0, no downgrading is granted.

4.3 The n-1 Attack

The most important verification test is the *n-1 attack*: given the aggregate output and all but one of the inputs, can the missing input be recovered?

For a **mean** of n values: yes, trivially. The mean alone does not qualify for downgrading. However, the mean *with a minimum population size* does, because in practice the attacker does not have $n - 1$ of the inputs. The verification protocol accounts for this:

- If $T_\downarrow$ is vulnerable to the n-1 attack, downgrading is granted only when the population size at query time meets or exceeds $n_{\min}$.
- $n_{\min}$ is determined during verification: it is the smallest n for which the bounded inference test shows no individual input constrained to less than 50% of the domain, given only the output (not n-1 inputs).
- This threshold is recorded as part of the transform's registration metadata and enforced at query time.

4.4 Population Size Enforcement

Definition 4 (Minimum Population Threshold)**.** *A downgrading transform $T_\downarrow$ carries a minimum population threshold $n_{\min}$. At query time, if the number of input records $|P| < n_{\min}$,*

the transform executes but its output retains the source classification level. The downgrade is applied only when $|P| \geq n_{\min}$.

This is critical. A "mean" of 2 values is nearly as sensitive as the values themselves. A mean of 10,000 values reveals almost nothing about any individual. The population threshold formalizes this intuition.

5. THE UNVERIFIABLE INFORMATION LOSS PRINCIPLE

5.1 Statement

Theorem 1 (No Downgrade Without Verification)**.** *A transform T whose information destruction cannot be empirically measured by the verification protocol retains the source classification level on its output. Claimed but unverifiable information loss does not justify downgrading.*

This is the central safety property of the system. It means:

1. A custom WASM transform that claims to "anonymize" data but whose behavior cannot be characterized by the verification tests does not receive downgrading.
2. A transform that operates on data types for which the verification protocol cannot generate meaningful synthetic populations does not receive downgrading.
3. A transform whose output format is opaque (e.g., a binary blob) and cannot be analyzed for reconstruction resistance does not receive downgrading.

The burden of proof is on the transform: it must *demonstrably* destroy information, not merely claim to.

5.2 Rationale

In classical information security, downgrading decisions are made by trusted human reviewers who examine specific outputs and judge whether they are safe for release at a lower level. This process is slow, subjective, and does not scale.

Automated downgrading replaces human judgment with empirical testing. But empirical testing has limits: it can only verify properties it can measure. A transform that produces output in a format the test harness cannot analyze, or that destroys information in a way that cannot be detected by reconstruction attacks, falls outside the verification boundary.

Rather than guessing about such transforms, the system takes the conservative position: no verification, no downgrade. This ensures that the downgrading mechanism can never be exploited by a transform that appears to destroy information but actually preserves it in a non-obvious way (steganography, encoding, etc.).

6. PUBLIC VISIBILITY AND AUDITABILITY

6.1 Transform Registry

All downgrading transforms are registered in FoldDB's Universal Transform Registry (UTR) with:

- The transform's source code or WASM bytecode.
- Its content-addressed hash (immutable identifier).
- The claimed aggregation class (mean, median, count, etc.).
- The verification results: test populations used, reconstruction attack outcomes, assigned score, granted downgrade level.
- The minimum population threshold $n_{\min}$.

This registration is public. Any data owner whose data will be subject to a downgrading transform can inspect:

1. What the transform computes (source code is visible).
2. How it was tested (verification results are recorded).
3. What downgrade it received and why (score breakdown is available).
4. How many records are required for the downgrade to apply ($n_{\min}$).

6.2 Downgrade Chain Visibility

When a fold's output carries a downgraded classification, the full provenance chain is recorded:

$$\ell_{\text{source}} \xrightarrow{T_1, n_1} \ell_1 \xrightarrow{T_2, n_2} \ell_2 \xrightarrow{T_3, n_3} \ell_{\text{output}}$$

Each link records the transform applied, the population size at evaluation time, and the resulting classification. This chain is attached to the output metadata, enabling any consumer of the downgraded data to trace it back to its source classification and verify that each downgrading step was legitimate.

6.3 Re-verification

If a data owner updates the verification protocol (e.g., adds new reconstruction attacks), previously registered transforms can be re-verified. If a transform fails re-verification, its downgrading privilege is revoked for future queries. Outputs already produced retain their classification (no retroactive reclassification), but the audit trail records the re-verification failure.

7. INTEGRATION WITH FOLDDB ACCESS CONTROL

7.1 Interaction with the Four Access Control Layers

Downgrading transforms interact with FoldDB's conjunctive access control as follows:

Trust Distance: A downgraded output may have a wider trust distance threshold than its source. For example, individual salary records (level 3, R_1) may produce a department average (level 1, R_5), accessible to users at greater trust distance.

Capabilities: Downgraded outputs can have independent capability constraints. A capability to access the average does not grant capability to access individual records.

Security Labels: The downgrading mechanism modifies the label lattice by introducing verified downward edges. These edges exist only for specific registered transforms and are not general-purpose. The lattice remains strict for all non-verified transforms.

Payment Gates: Downgraded outputs may have lower (or zero) payment requirements, reflecting their reduced sensitivity.

7.2 Query-Time Enforcement

At query time, when a downgrading transform is evaluated:

1. The execution engine counts the number of input records that pass all four access control checks.
2. If the count $\geq n_{\min}$, the transform executes and the output receives the downgraded classification.
3. If the count $< n_{\min}$, the transform executes but the output retains the source classification. The caller may or may not have access at the source level.

4. If the count is 0 (all records denied), the fold returns Nothing per standard monadic failure semantics.

This means the population threshold is enforced against *authorized* records, not total records. An attacker cannot inflate the population count by injecting records they control.

7.3 Composition of Downgrades

Downgrading transforms compose. If transform T_1 downgrades from level 4 to level 2, and transform T_2 downgrades from level 2 to level 0, the composed chain $T_2 \circ T_1$ produces level 0 output from level 4 input — but only if both transforms independently pass verification and both population thresholds are met at query time.

Each step in the chain is independently verified. There is no transitive trust: the fact that T_1 is verified does not affect the verification of T_2.

8. EXAMPLES

8.1 Salary Analytics

A company stores individual employee salaries at classification (3, `financial`). They want to publish department-level salary statistics.

Transform: T_{median} computes the median salary per department.

Verification: The median is not vulnerable to the n-1 attack (knowing n-1 values and the median does not uniquely determine the missing value for $n > 2$). Bounded inference test shows no individual constrained to less than 25% of domain for $n \geq 5$. Distinguishability ratio is 1.02 for $n = 20$. Score: 3.

Result: T_{median} is granted downgrade of 2 levels with $n_{\min} = 5$. Department median salaries are classified at (1, `financial`), accessible to users at trust distance ≤ 5 instead of ≤ 1.

Edge case: A department with 3 employees does not meet $n_{\min} = 5$. The median is still computed but classified at (3, `financial`), same as individual salaries.

8.2 Health Data Histograms

A research institution stores individual patient blood pressure readings at (4, `medical`). They want to publish population health statistics.

Transform: T_{hist} produces a histogram with 10 mmHg bins.

Verification: Each bin contains a count. Exact inversion: not possible (count does not reveal individual values). Bounded inference: an individual's value is constrained to a 10 mmHg range (the bin width). For typical blood pressure ranges (80–200 mmHg), this is $10/120 \approx 8.3\%$ of the domain. Score: 3 for the count per bin. However, if any bin contains fewer than $n_{\min}$ individuals, that bin's count is suppressed (replaced with "$< n_{\min}$").

Result: T_{hist} granted downgrade of 1 level (distribution class, max -1) with $n_{\min} = 10$ per bin. Histogram classified at (3, `medical`). Bins with fewer than 10 individuals are suppressed.

8.3 Non-Downgrading Example

A developer writes a WASM transform that "anonymizes" user profiles by removing the name field but keeping age, zip code, and gender.

Verification: The n-1 attack is not applicable (no aggregation). Distinguishability: given an output profile (age 34, zip 98101, gender M), there may be very few individuals matching this combination. Bounded inference: age is exactly preserved, zip is exactly preserved. Score: 0.

Result: No downgrade. Output classification remains identical to input. The transform is registered but carries no downgrading privilege.

9. FORMAL PROPERTIES

Property 1 (Conservative Safety)**.** *If the verification protocol cannot determine whether a transform destroys information, the transform receives no downgrade. False negatives (denying a legitimate downgrade) are preferred over false positives (granting an illegitimate one).*

Property 2 (Population-Dependent Classification)**.** *The classification of a downgrading transform's output is a function of both the transform and the population size at query time. The same transform may produce differently-classified outputs depending on how many records are available.*

Property 3 (Monotonicity of Aggregation)**.** *For approved aggregation classes, information destruction is monotonically non-decreasing in population size. Larger populations produce less sensitive aggregates (or equally sensitive, never more sensitive).*

Property 4 (Non-Transitivity of Trust)**.** *Verification of one transform in a chain does not affect verification of another. Each downgrading step is independently verified, independently enforced, and independently auditable.*

Property 5 (Lattice Integrity)**.** *For all non-downgrading transforms, the standard lattice constraint $\ell_{in} \sqsubseteq \ell_{out}$ is preserved. Downgrading transforms introduce controlled exceptions, not a general relaxation of the lattice.*

10. RELATED WORK

Differential Privacy (Dwork et al., 2006) provides mathematical guarantees about information leakage from aggregate queries by adding calibrated noise. Our approach is complementary: differential privacy bounds what an adversary can learn; our verification protocol bounds what classification level the output receives. The two can be composed — a differentially private mean would score higher on the verification protocol than a raw mean, potentially receiving a greater downgrade.

k-Anonymity (Sweeney, 2002) ensures each record is indistinguishable from at least $k - 1$ others. Our minimum population threshold serves a similar purpose but applies to the aggregation input rather than individual records. k-anonymity can be used as a preprocessing step before a downgrading transform.

Bell–LaPadula Model (1973) establishes the "no write down" property that our base system enforces. Our contribution is a principled mechanism for controlled declassification, which Bell–LaPadula does not address.

Declassification in Language-Based Security (Sabelfeld and Myers, 2003) identifies dimensions of declassification: what information is released, who can release it, where in the code, and when. Our system addresses the "what" dimension through empirical verification and the "who" dimension through FoldDB's existing access control (only fold owners can register downgrading transforms on their data).

11. CONCLUSION

The tension between data sensitivity and data utility is not binary. Between "fully classified" and "fully public" lies a spectrum of aggregation levels, each with a verifiable degree of information destruction. FoldDB's classification downgrading mechanism makes this spectrum explicit, auditable, and automatically enforced.

The core principle is simple: **if you can prove that a transform destroys information, the output can be classified lower than the input. If you cannot prove it, it cannot.** This moves downgrading decisions from subjective human judgment to empirical measurement, while maintaining the conservative safety property that unverifiable claims receive no benefit.

By integrating downgrading into FoldDB's existing transform and access control infrastructure — publicly registered transforms, content-addressed identifiers, conjunctive policy

enforcement, monadic failure semantics — we enable a new class of data sharing: sensitive data can contribute to public statistics without ever being exposed, and the entire chain from raw data to public aggregate is visible, verifiable, and auditable.

FIGURES

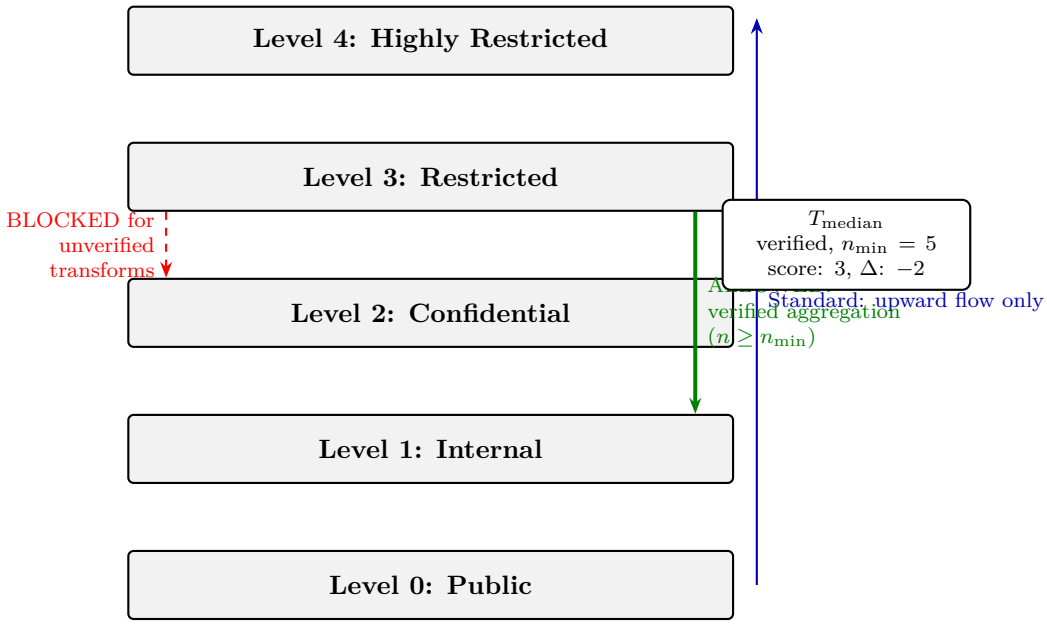

Figure 1: **Classification Downgrading Overview.** Standard transforms enforce upward-only information flow (blue). Unverified transforms are blocked from downgrading (red). Verified aggregation transforms with sufficient population size may produce lower-classified outputs (green).

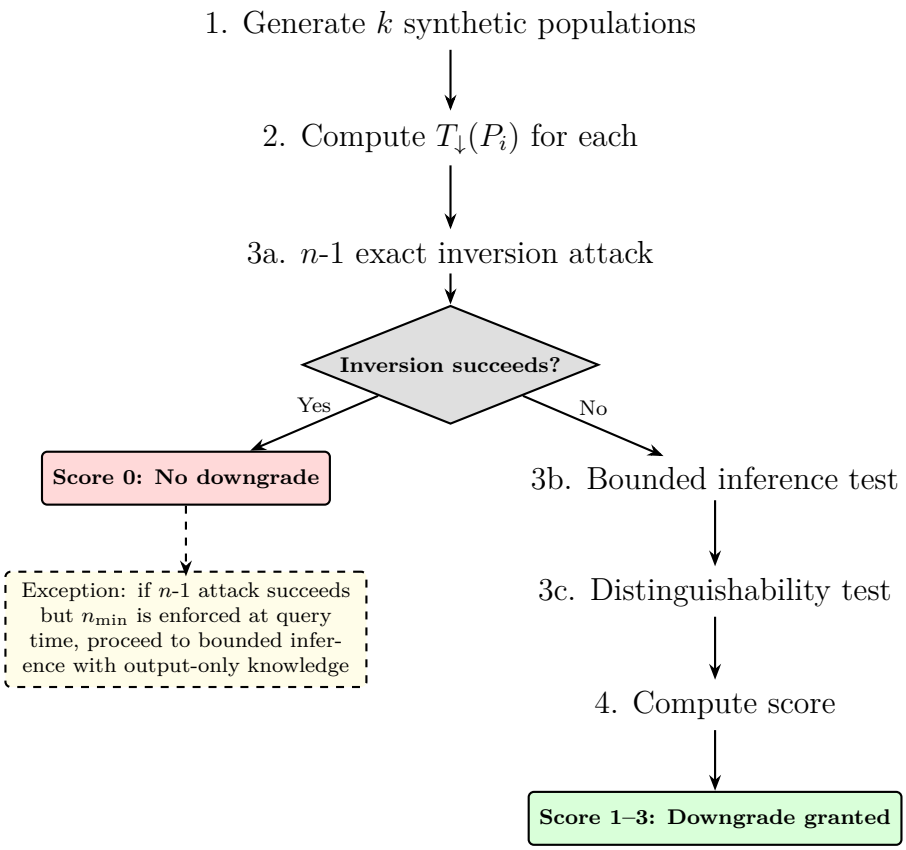

Figure 2: **Verification Protocol.** Transforms are tested against reconstruction attacks at registration time. Transforms vulnerable to exact inversion receive no downgrade (unless protected by population threshold). Surviving transforms are scored based on bounded inference and distinguishability.

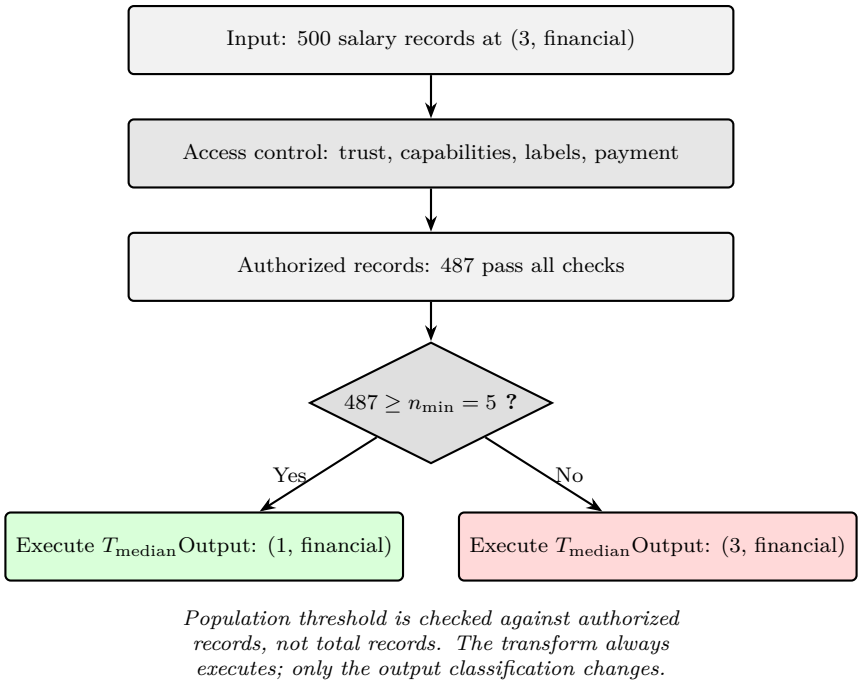

Figure 3: **Query-Time Enforcement.** The population threshold is evaluated after access control filtering. If enough records are authorized, the output receives the downgraded classification. Otherwise, the output retains the source classification.

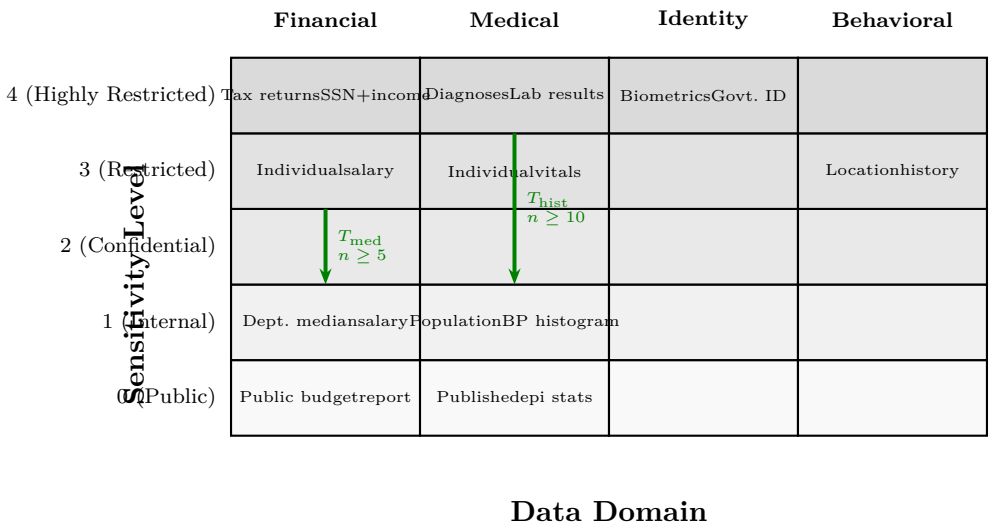

Figure 4: **Two-Dimensional Classification Space.** Data is classified by sensitivity level (vertical) and domain (horizontal). Downgrading transforms (green arrows) move data downward within a domain. Cross-domain flow requires explicit authorization and is never automatic.